

GESTION DE LA MÉMOIRE CENTRALE

M2 Secil : KINN9HU
Pascal SAINRAT



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER

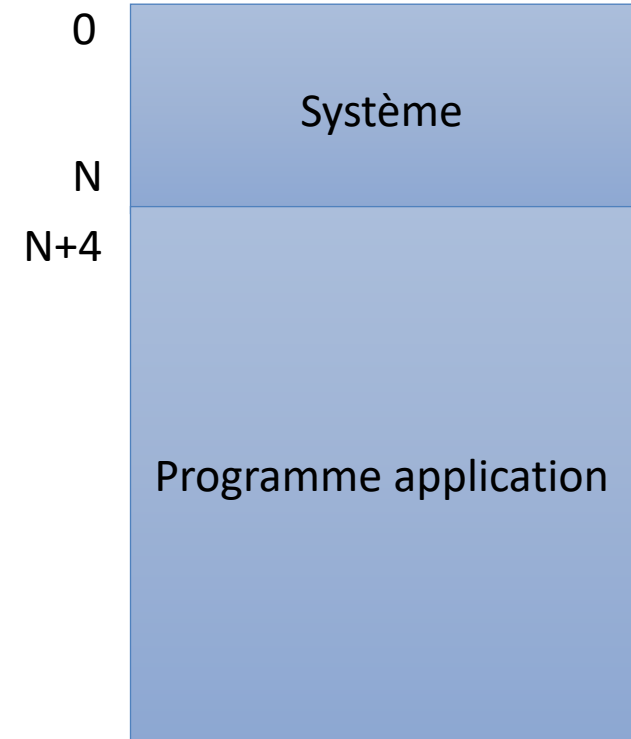


Faculté
Sciences
et Ingénierie

Monoprogrammation

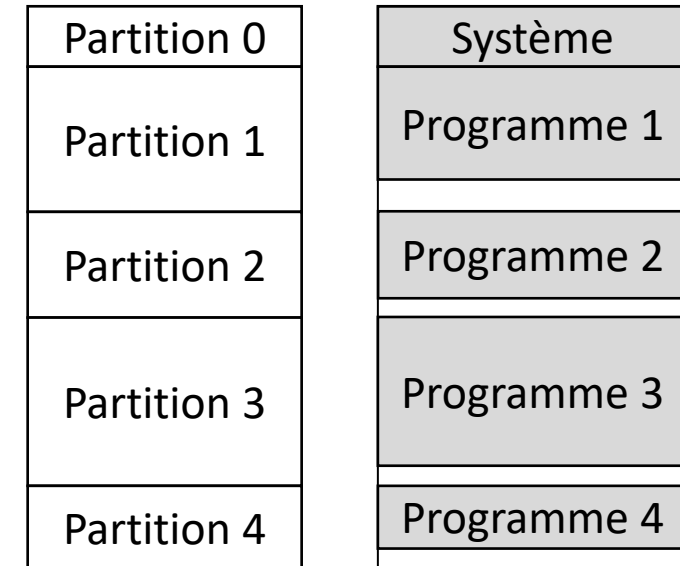
- Partager la mémoire entre le programme d'application et la partie du système résidant en mémoire
- S'il n'y a pas assez de place pour toute l'application, l'application doit être découpée en modules (overlays) qui doivent pouvoir se succéder dans la mémoire.

La nature a horreur du vide !
- Aujourd'hui essentiellement pour des systèmes embarqué simples
 - Le "système" est en ROM
 - Le programme et ses données sont en Flash
 - Le programme est en ROM ou en Flash, les données sont en SRAM ou DRAM selon la quantité



Partitions de taille fixe

- Idée la plus simple
- La mémoire est découpée en partitions fixes de longueur unique ou variable une fois pour toutes
- Tri des processus en attente par taille
- Mauvaise utilisation de la mémoire car la taille des processus ne correspond pas forcément à celle des partitions
- Pas d'allocation dynamique bien sûr



Partitions de taille variable

Système
Programme 1
Programme 2
Programme 3
Programme 4

Chargement initial

A chaque processus est attribué une partition correspondant à sa taille

Système
Programme 1
Programme 3
Programme 4

Après un certain temps

P2 fini
Pas de place pour P5

Système
Programme 1
Programme 3
Programme 4
Programme 5

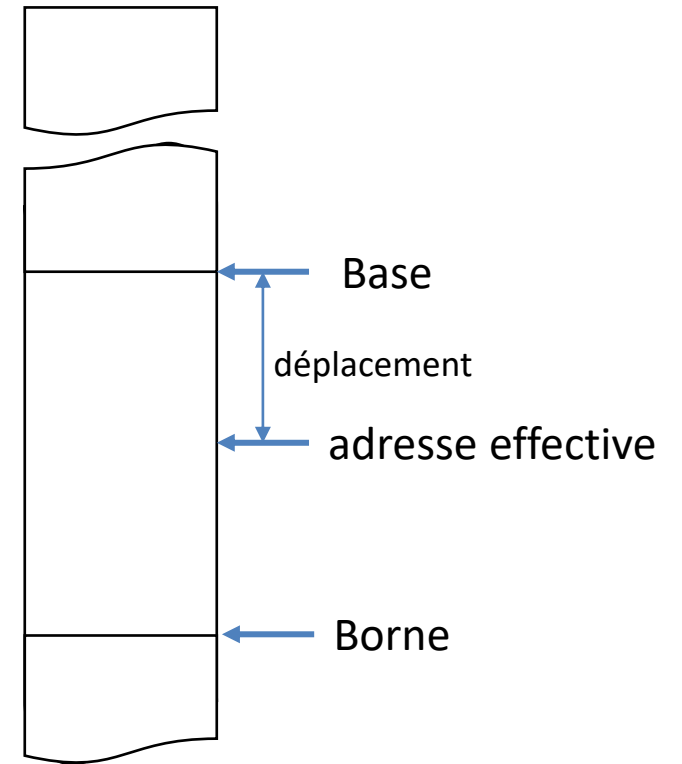
Après retassement par réallocation dynamique

Translation dynamique et protection

- Protection : tout accès d'un processus a lieu dans la partition de la mémoire qui lui a été allouée (entre adresse de base et adresse max (bound))
- Dispositifs pour
 - Mémoriser l'adresse de base et l'adresse max (borne)
 - Calculer l'adresse effective (adresse de base + déplacement)
 - Vérifier que l'adresse effective est entre la base et la borne

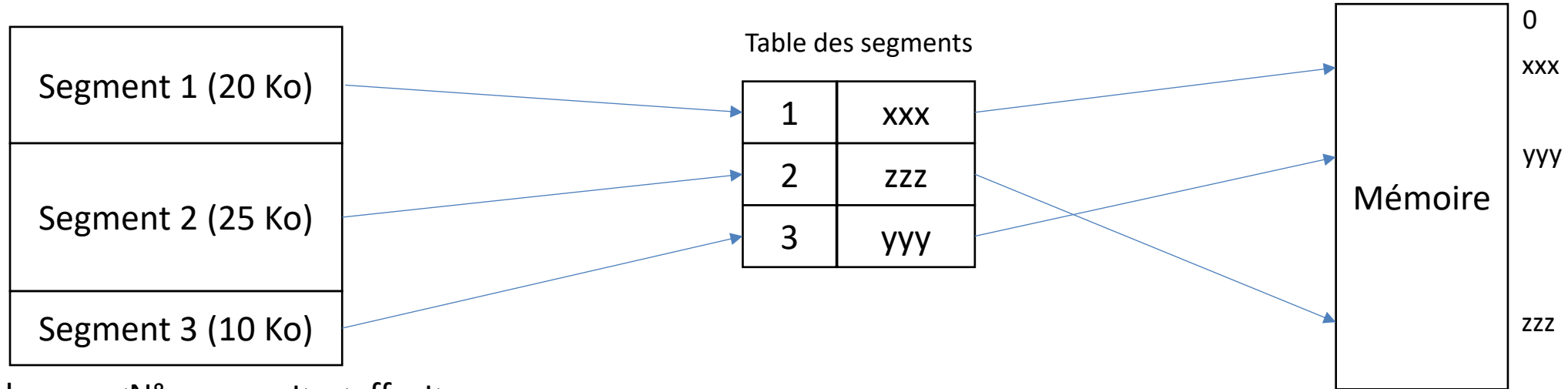
Tassement possible mais long

- Zone mémoire allouée à un processus d'un seul tenant
 - Fractionnement programme
 - Partitionnement de la mémoire
 - Allocations de plusieurs partitions de la mémoire à un processus



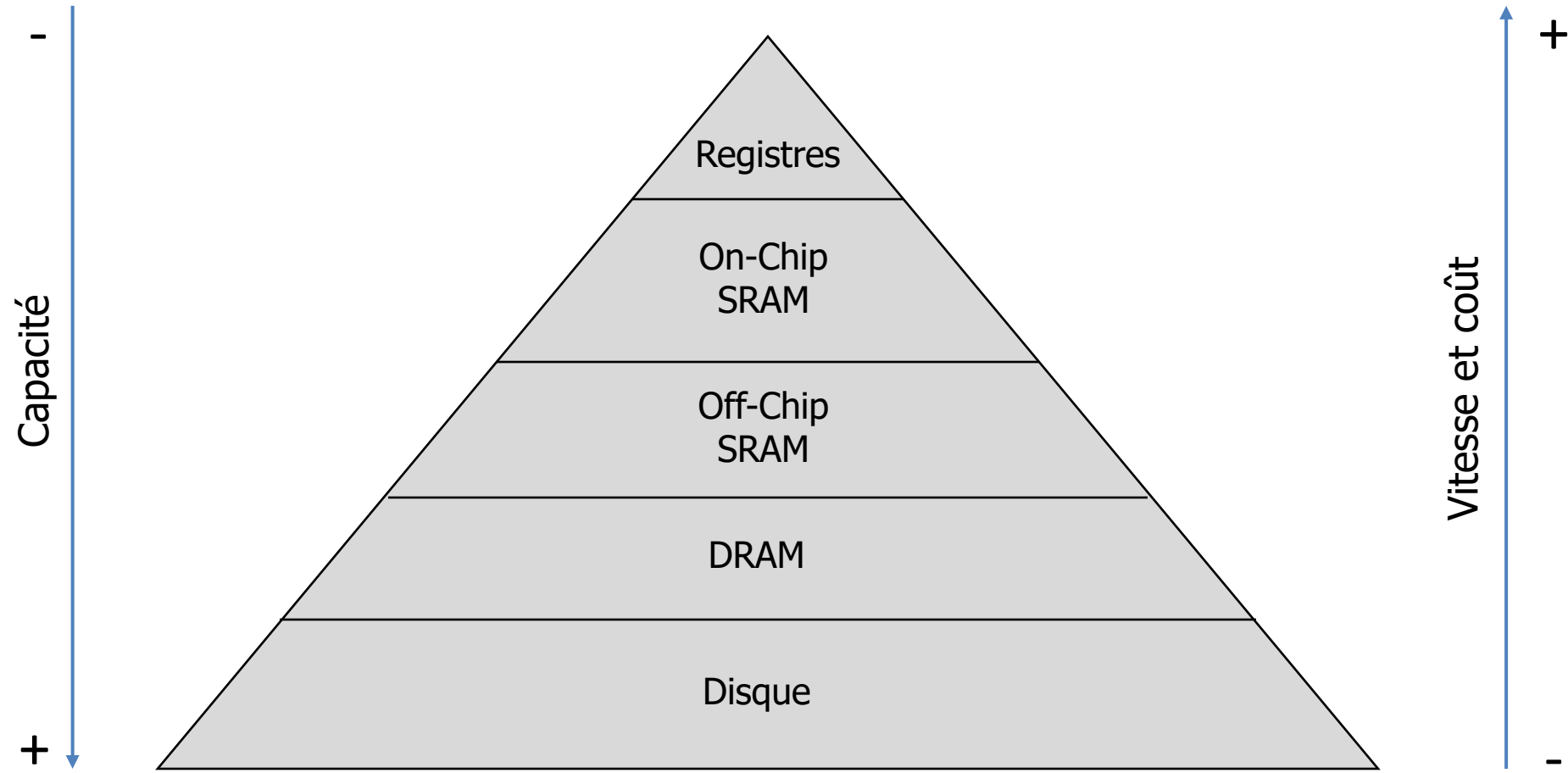
Segmentation

- Division du programme en segments (procédure, bloc de données, ...)
- Plusieurs espaces d'adressage séparés
- Les segments sont plus petits et donc plus aptes à occuper la mémoire sans tassements
- Le système tente de placer au mieux chaque segment dans chaque espace libre
- Il conserve, pour chaque processus, une table de segments mémorisant où chaque segment est placé en mémoire (et l'adresse de fin ou la taille) pour la protection.



adresse=<N° segment>.<offset>

Éléments de mémorisation

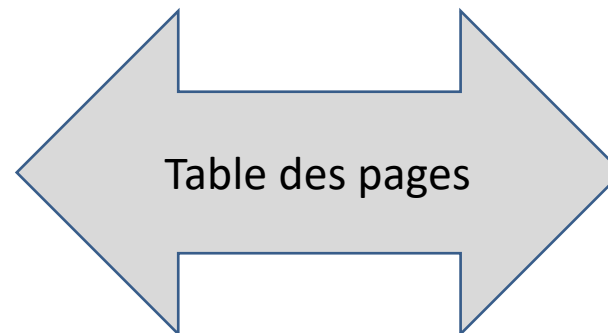


Notion de mémoire virtuelle

Programme	Mémoire physique
Adresses virtuelles	Adresses réelles

Chaque processus a accès à une mémoire dont la taille est indépendante de la taille de la mémoire physique. La taille de l'espace est fonction de la taille des pointeurs elle-même dépendant (aujourd'hui 32 ou 64 bits mais on commence à travailler sur des processeurs avec un adressage de 128 bits).

Le système d'exploitation est en charge d'allouer de la mémoire physique à un processus. Si la place en mémoire physique n'est pas assez grande par rapport à la mémoire demandée par le processus, le système utilise le disque et alloue la mémoire centrale selon les besoins.



Pagination

- Les deux espaces (virtuel et réel) sont divisés en pages de même taille.
 - Dans l'espace réel, on parle de cadres *frames* destinés à recevoir des pages.
 - Les pages des processus sont allouées par le système d'exploitation au fur et à mesure des besoins.
-
- Une page est-elle en mémoire et où ?
 - Conversion des adresses virtuelles en adresses réelles ?
 - Quelle page remplacer si nécessaire ?
 - La page à remplacer a-t-elle été modifiée ?
-
- Toute l'information (et plus) est présente dans la table des pages du processus, elle-même placée dans un cadre de page

Table des pages

1	1	xxx	uuu	1	111
2	0	yyy			
3	1	zzz	vvv	0	100

Numéro de
page virtuelle

Page présente
en DRAM ou
non ?

Emplacement
sur disque

Numéro de
cadre de page
réel

Page modifiée
ou non ? Write-back !

Droits

Défaut de page *page fault*

- Pas de page allouée pour un emplacement demandé par le processeur
- Exception : sauvegarde contexte
- Appel gestionnaire de défaut de page du système
 - Trouver un cadre (possiblement en éjectant une page)
 - Lancer le chargement de la nouvelle page depuis le disque
 - Passer à une autre tâche prête à être exécutée
 - Interruption quand le chargement de page est terminé
 - Restauration contexte

Différence avec le cache où, en cas d'échec, le matériel se débrouille pour charger le bloc demandé

Taille(s)

- Taille d'une page : quelques Ko (souvent 4 Ko) pour ne pas avoir trop d'espace inoccupé
- Taille de la table des pages
 - $4 \cdot 2^9 / 4 \cdot 2^3 = 2^6 = 1 \text{ M}$ entrées (1 entrée = 4 octets)
 - Utilisation de tables de page multi-niveaux
 - Table de pages hachée
 - Table de pages inversée

Remplacement

- FIFO, random
- LRU pas possible car trop d'entrées
 - Approximé par des bits de référencement (mis à 1 quand on accède à une page, raz par l'OS périodiquement)
- Charge à l'OS de choisir selon une politique donnée

Table de pages multi-niveaux

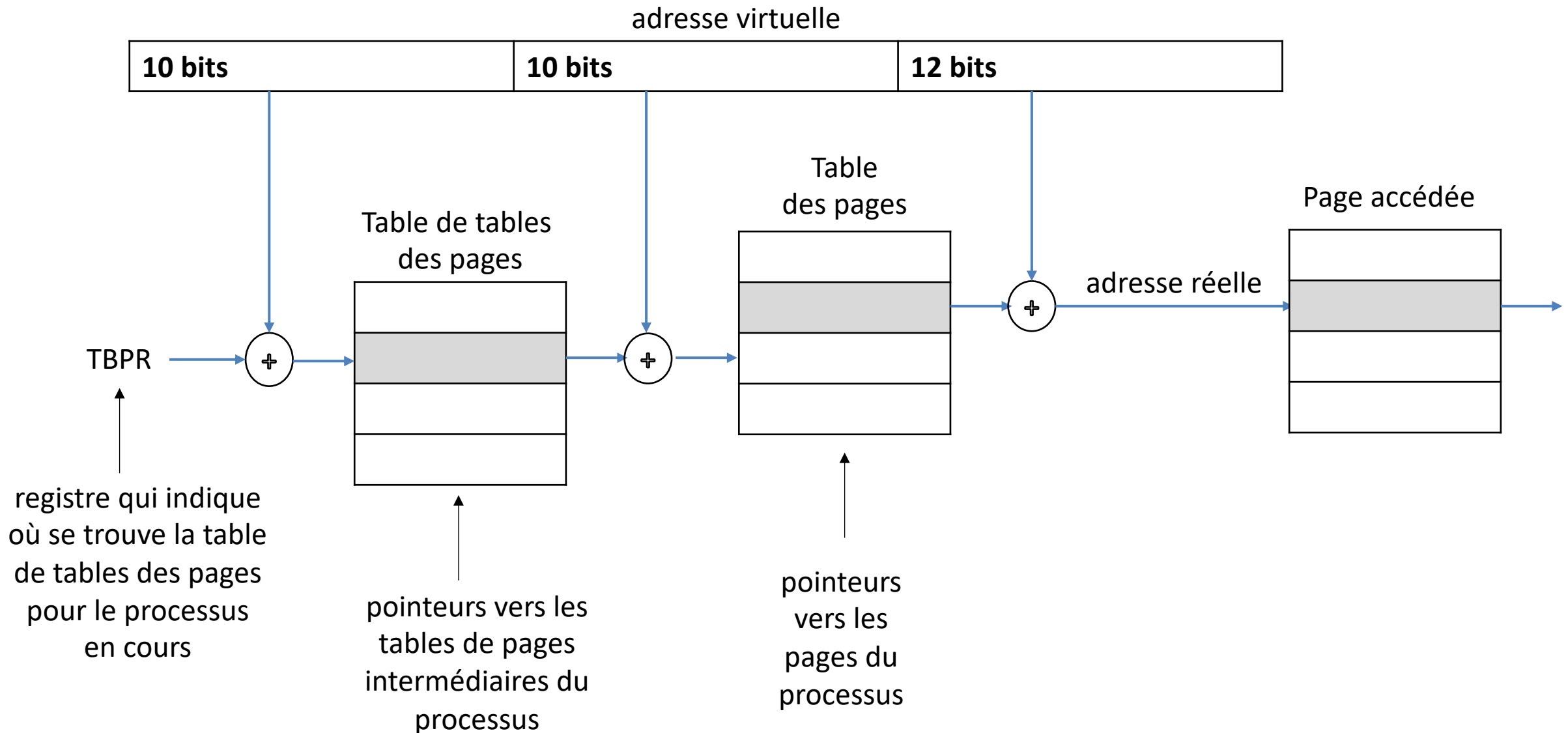


Table de pages hachée

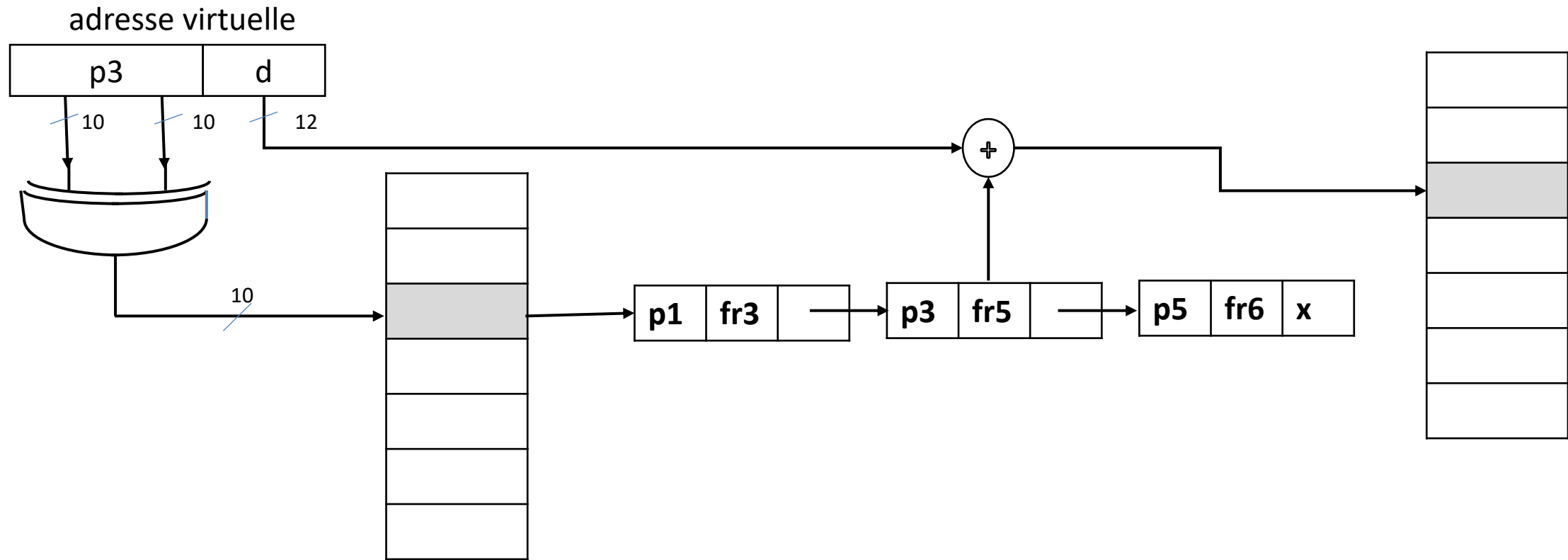
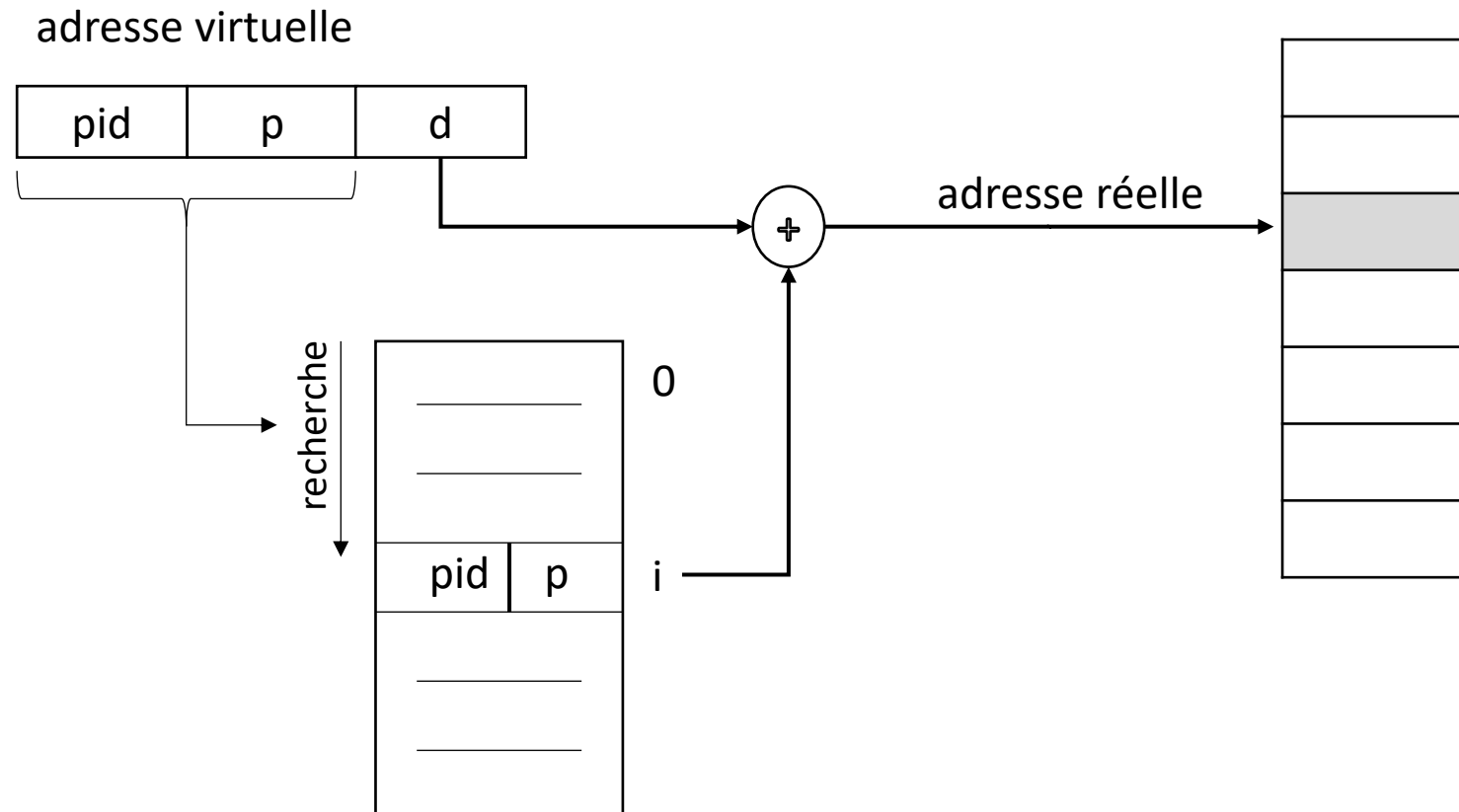


Table de pages inversée

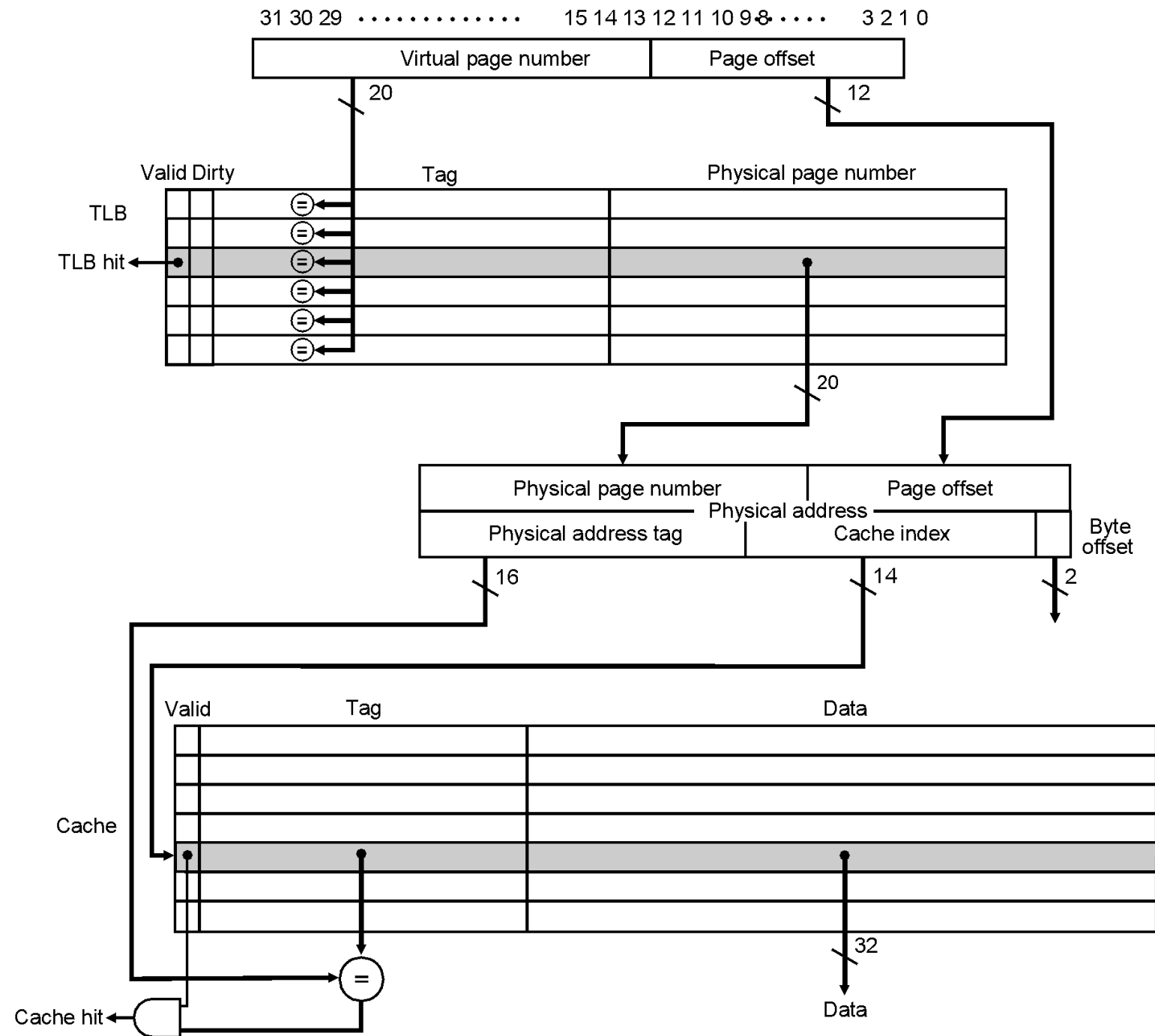
- Table des pages de la mémoire physique et non une table par processus



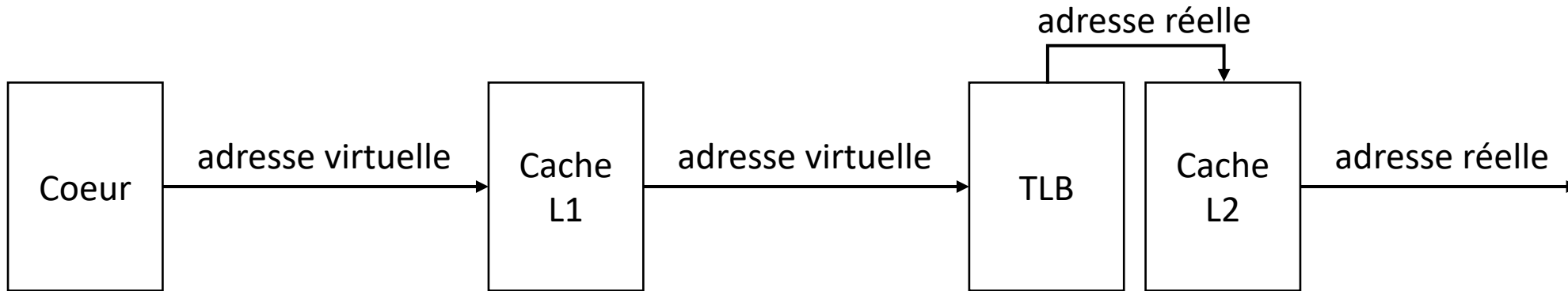
Performance

- Traduction d'adresse
 - Nécessite 2 accès à la mémoire (1 à la table des pages, 1 à la donnée)
 - Voire plus si table multi-niveaux ou traversée table inversée
 - Même si la table des pages (ou une partie de celle-ci) est dans la mémoire cache, cela reste long
- Utilisation d'un cache spécial pour la table des pages
 - TLB (Translation Look-aside Buffer)
 - Exploite la localité spatiale et temporelle (comme les caches de données ou d'instructions)

TLB



TLB et hiérarchie mémoire caches



Inclusion stricte L1 dans L2 + pas de synonymes dans le L2 (en adresse réelle) = pas de synonymes dans le L1

L1 en adresses virtuelles : contenu L1 propre à un processus

L1 write-through : purge cache instantanée quand changement de processus

TLB et hiérarchie mémoire caches

